

Project 3: Link Analysis using PageRank

Algorithms for Massive Data

Master in Data Science for Economics

Università degli Studi di Milano

Student name: Srishti Kansra

Student ID: 49483A

Academic year: 2025/26

Submitted to Prof. Dario Malchiodi

Academic Integrity Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

Contents

1	Dataset	3
1.1	Source	3
1.2	Subset Used	3
1.3	Fields Used	3
2	Data Organisation	3
3	Pre-processing	4
3.1	Author Name Normalisation	4
3.2	Deduplication Within Papers	4
3.3	Isolated Node Preservation	4
3.4	Limitations	5
4	Algorithms and Implementations	5
4.1	PageRank: Theoretical Background	5
4.2	Dead Ends and Spider Traps (RU 5.1.3–5.1.5)	5
4.3	Power-Iteration Implementation (Single-Machine)	6
4.4	MapReduce Formulation in Spark (RU 5.1.3, Lecture 22/01)	6
5	Scalability	7
5.1	Theoretical Complexity	7
5.2	Empirical Scalability Experiment	7
5.3	Scaling to the Full arXiv Snapshot	7
6	Experiments	8
6.1	Network Statistics	8
6.2	PageRank Computation and Convergence	8
6.3	Top-20 Authors by PageRank	8
6.4	Degree vs. PageRank Comparison	8
6.5	Largest Connected Component	8
6.6	Network Visualisations	9
6.7	Spark MapReduce PageRank	9
7	Results and Discussion	9
7.1	Network Structure	9
7.2	Top-Ranked Authors	9
7.3	Degree vs. PageRank Divergence	10
7.4	Convergence Behaviour	10
7.5	Spark Validation	10
7.6	Limitations and Future Directions	10
	References	11

1. Dataset

1.1 Source

The project uses the **arXiv dataset**, published by Cornell University under the Public Domain licence and available on Kaggle (identifier: **Cornell-University/arxiv**). The dataset is a newline-delimited JSON file where each record represents one arXiv paper and contains metadata including title, abstract, authors, subject categories, and publication dates. The full snapshot contains approximately 2 million papers spanning all scientific disciplines represented on arXiv.

1.2 Subset Used

The full snapshot is too large to process on a single Colab instance without out-of-memory errors. A global configuration flag controls whether the full dataset or a subsample is used:

```
USE_FULL_DATASET = False    # set True to use the full 2M-paper
                             snapshot
NROWS = None if USE_FULL_DATASET else 50_000
```

With the flag set to **False**, the first 50,000 papers are loaded. This subset is large enough to produce a meaningful collaboration network while keeping runtimes practical in a Colab environment. All code paths are independent of this flag, so the full pipeline runs unchanged on the complete dataset.

1.3 Fields Used

Only the following field is used from each paper record:

- **authors_parsed**: a structured list of [last, first, suffix] triples representing the paper's authors. This field is preferred over the raw **authors** string because it avoids the ambiguities of regex-based name parsing across the diverse formatting conventions in the arXiv corpus.

2. Data Organisation

The core data structure is an **undirected weighted graph** $G = (V, E)$ constructed from the set of papers:

- **Nodes** V : one node per unique author name, as extracted from **authors_parsed**.
- **Edges** E : an edge (u, v) exists between two authors if they appear together in the **authors_parsed** field of at least one paper.
- **Edge weight**: the number of papers on which the two authors co-appear. Repeated collaborations increase the weight.

For a paper with k authors, all $\binom{k}{2}$ pairs among the k authors are connected (a clique of size k). This approach is standard in the co-authorship network literature [4]. The graph

is constructed by the function:

```
def build_graph(data: pd.DataFrame) -> nx.Graph
```

which takes a DataFrame of arXiv records and returns a NetworkX Graph. This function is encapsulated so it can be called identically for the main graph and for each dataset fraction in the scalability experiment.

On the 50,000-paper subset the resulting graph contains approximately **85,000 nodes** and **500,000 edges**. The largest connected component accounts for a large majority of both nodes and edges, consistent with the well-known small-world property of scientific collaboration networks.

3. Pre-processing

3.1 Author Name Normalisation

Author names are extracted from `authors_parsed` using:

```
def author_name(triple):  
    last = triple[0].strip()  
    first = triple[1].strip()  
    return f"{first}_{last}".strip()
```

This produces consistent *First Last* format. The structured field is preferred over the raw `authors` string, which mixes delimiters and formatting conventions across different papers.

3.2 Deduplication Within Papers

For each paper, the set of author names is deduplicated before constructing edges. This prevents a paper from generating self-loops or duplicate edges in cases where the same name appears twice in the parsed field (which can occur due to OCR artefacts in older records).

3.3 Isolated Node Preservation

Single-author papers produce isolated nodes (degree 0) in the graph. These are retained rather than discarded to maintain full coverage of the author population. In the PageRank computation, isolated nodes receive only the teleportation contribution and emit nothing during edge propagation — this is the correct behaviour and is discussed further in Section 4.3.

3.4 Limitations

Two pre-processing limitations are acknowledged:

- **No name disambiguation:** “J. Smith” and “John Smith” are treated as distinct nodes. This inflates the node count and fragments some authors’ true collaboration networks. ORCID-based disambiguation would be required for a production system.
- **No category filtering:** papers from all arXiv subject categories are included. Ranking results therefore reflect cross-disciplinary collaboration patterns rather than a single field.

4. Algorithms and Implementations

4.1 PageRank: Theoretical Background

PageRank [2] assigns an importance score to each node in a graph by simulating the behaviour of a *random surfer*. At each step the surfer either:

- with probability α (the *damping factor*): follows a random neighbour, with probability proportional to the edge weight connecting the current node to that neighbour;
- with probability $1 - \alpha$: *teleports* to a uniformly random node in the graph.

The stationary distribution of this Markov chain is the PageRank vector $\mathbf{r}$, satisfying:

$$r(u) = \frac{1 - \alpha}{|V|} + \alpha \sum_{v \in N(u)} \frac{w(v, u)}{\text{out}(v)} \cdot r(v) \quad (1)$$

where $N(u)$ is the set of neighbours of u , $w(v, u)$ is the weight of edge (v, u) , and $\text{out}(v) = \sum_k w(v, k)$ is the total outgoing weight of v . We use $\alpha = 0.85$, the value proposed in the original paper.

4.2 Dead Ends and Spider Traps (RU 5.1.3–5.1.5)

Two structural pathologies must be handled in any correct PageRank implementation:

- **Dead ends:** nodes with no outgoing edges. A random surfer at a dead end has nowhere to go, causing probability mass to *leak* out of the system — the score vector no longer sums to 1. In our undirected co-authorship graph, dead ends are the isolated degree-0 nodes produced by single-author papers.
- **Spider traps:** groups of nodes whose only outgoing edges point back within the group. A surfer trapped inside accumulates all reachable probability mass indefinitely, giving those nodes artificially inflated scores.

Both pathologies are resolved simultaneously by the **teleportation term** $(1 - \alpha)/N$: at every step, with probability $1 - \alpha = 0.15$, the surfer jumps to a uniformly random node regardless of the current node’s edges. Dead-end mass is redistributed across the graph; spider-trap mass can escape via teleportation.

After adding teleportation, the transition matrix is:

- **Stochastic:** rows sum to 1 (no mass leaks at dead ends).
- **Aperiodic:** teleportation breaks all periodic cycles.
- **Irreducible:** every node is reachable from every other node via teleportation.

By the Perron–Frobenius theorem, these three properties guarantee that a unique stationary distribution exists and that power iteration converges to it from any starting vector (RU 5.1.2).

4.3 Power-Iteration Implementation (Single-Machine)

The function `pagerank_power(G, alpha, max_iter, tol)` implements power iteration from scratch, following the formulation in Equation 1:

1. Pre-compute the total outgoing weight $\text{out}(v)$ for each node v .
2. Initialise the score vector uniformly: $r(v) = 1/N$ for all v .
3. At each iteration, compute a new score vector $\mathbf{r}_{\text{new}}$: each node v receives the teleportation contribution $(1 - \alpha)/N$ plus, for each neighbour u , the propagated contribution $\alpha \cdot [w(u, v)/\text{out}(u)] \cdot r(u)$.
4. Compute the L1 error: $\text{err} = \sum_v |r_{\text{new}}(v) - r(v)|$.
5. If $\text{err} < \text{tol}$, stop. Otherwise set $\mathbf{r} = \mathbf{r}_{\text{new}}$ and repeat from step 3.

The function returns both the final score dictionary and the per-iteration L1 error list, enabling the convergence curve to be plotted. The result is validated against NetworkX's reference implementation `nx.pagerank()`: the maximum absolute difference across all nodes is on the order of 10^{-6} (within the convergence tolerance), and both produce identical top-5 rankings.

4.4 MapReduce Formulation in Spark (RU 5.1.3, Lecture 22/01)

For datasets too large to fit in a single machine's memory, each power-iteration step maps directly onto a MapReduce job. The formulation follows RU 5.1.3:

- **MAP step:** for each source node u with current score $r(u)$, emit $(v, r(u) \cdot w(u, v)/\text{out}(u))$ for each neighbour v .
- **REDUCE step:** for each destination node v , collect all incoming contributions and compute:

$$r_{\text{new}}(v) = \frac{1 - \alpha}{N} + \alpha \sum_{\text{contributions}} c_v$$

This formulation is *embarrassingly parallel*: each node's update depends only on its neighbours' scores from the previous iteration, so the map step runs independently across all partitions simultaneously.

In PySpark, the MAP step is expressed as a `join + groupBy("dst") + agg(F.sum(...))` on a normalised edge DataFrame. Dead-end nodes (those that receive no contributions) are handled via a left join with the full node list, with missing contributions replaced by the teleportation floor using `F.coalesce()`.

Two engineering optimisations make the Spark loop viable on a 2-core Colab instance:

- **Scalar L1 error collection:** the convergence check uses a single `agg()` call (returning one scalar to the driver) rather than collecting the full rank DataFrame at each

iteration. This reduces per-iteration driver communication overhead from $O(N)$ rows to a single value.

- **DAG checkpointing:** the rank DataFrame is checkpointed to disk every 5 iterations to truncate Spark’s execution DAG, which would otherwise grow unboundedly across iterations and cause query-planning overhead to dominate runtime.

5. Scalability

5.1 Theoretical Complexity

The pipeline has two stages with the following complexities:

- **Graph construction:** $O(P \cdot k^2)$ where P is the number of papers and k is the average number of authors per paper (typically $k \leq 5$ in arXiv data). Inserting or incrementing a weighted edge in a hash-backed adjacency structure is $O(1)$ amortised.
- **PageRank (power iteration):** $O(I \cdot (N + E))$ per run, where I is the number of iterations to convergence, N is the number of nodes, and E is the number of edges. Because E grows approximately linearly with P (each paper adds at most $\binom{k}{2}$ edges), total runtime grows approximately linearly with dataset size.

5.2 Empirical Scalability Experiment

Wall-clock time is measured for the full pipeline (graph construction + PageRank convergence) across four fractions of the 50,000-paper subset: 25%, 50%, 75%, and 100%. Each fraction is timed three times and the mean $\pm$ standard deviation is reported to eliminate single-run noise artefacts.

Table 1: Scalability experiment results.

Dataset %	Nodes	Edges	Runtime (s)	Std (s)
25%	30,894	139,697	10.561	0.310
50%	52,770	239,029	20.323	3.884
75%	70,483	369,414	46.306	6.784
100%	85,318	508,460	48.304	2.300

Runtime grows approximately linearly with the number of edges, consistent with the theoretical $O(I \cdot (N + E))$ complexity. The mean-over-three-runs methodology eliminates the non-monotone artefact that appears in single-run measurements (caused by OS-level CPU scheduling noise on a 2-core Colab VM).

5.3 Scaling to the Full arXiv Snapshot

The full arXiv snapshot contains approximately 2 million papers. Extrapolating linearly from the 50,000-paper results, the single-machine pipeline would require on the order of 3–5 minutes on a standard laptop CPU. However, *memory* becomes the binding constraint before runtime: the full graph requires several gigabytes of RAM for the adjacency structure alone, making the single-machine version infeasible.

The Spark MapReduce implementation resolves this: edges and rank vectors are stored as distributed DataFrames, partitioned across the cluster. Scaling the cluster adds memory and compute proportionally. The same PySpark code that runs on the Colab VM in local mode runs unchanged on a multi-node cluster such as Google Dataproc, requiring only a change to the `SparkSession` master URL.

6. Experiments

6.1 Network Statistics

After constructing the co-authorship graph from 50,000 arXiv papers, the following statistics are computed: number of nodes and edges, average and maximum degree, graph density, and number of connected components with the size of the largest.

The degree distribution is visualised on both linear and log-log scales. The log-log plot reveals a heavy-tailed distribution, consistent with preferential-attachment growth models [1]: most authors have few collaborators, but a small number of hubs have hundreds.

6.2 PageRank Computation and Convergence

The custom `pagerank_power()` function is run on the full graph with $\alpha = 0.85$, `max_iter` = 100, and `tol` = 10^{-6} . The per-iteration L1 error is recorded and plotted on a logarithmic scale to visualise the convergence rate. The algorithm converges within 60 iterations, well below the `max_iter` ceiling of 100.

The result is validated against NetworkX's `nx.pagerank()` reference implementation. The maximum absolute difference across all nodes is on the order of 10^{-6} , and both produce identical top-5 rankings, confirming correctness of the custom implementation.

6.3 Top-20 Authors by PageRank

The 20 highest-ranked authors are extracted and displayed both as a table (alongside their degree for comparison) and as a horizontal bar chart. This directly answers the project objective of identifying influential researchers in the arXiv co-authorship network.

6.4 Degree vs. PageRank Comparison

To demonstrate that PageRank captures information beyond simple connectivity, a degree ranking and a PageRank ranking are computed for all authors and the absolute rank discrepancy is analysed. Authors with the largest discrepancy are reported. A log-log scatter plot of degree vs. PageRank score for all authors is produced to visualise the overall relationship.

6.5 Largest Connected Component

The size and edge coverage of the largest connected component is computed to quantify the graph's connectivity structure and contextualise the PageRank scores.

6.6 Network Visualisations

Two network visualisations are produced using spring-layout positioning:

- **Global view:** the top-20 PageRank authors and all of their direct collaborators. Top-20 nodes are coloured red; neighbour nodes are coloured blue.
- **Ego-network:** the direct collaboration neighbourhood of the top-ranked author (up to 100 neighbours), with the ego node highlighted in red.

6.7 Spark MapReduce PageRank

The Spark implementation is run on the same graph using the MapReduce formulation. It is capped at 20 iterations on the Colab VM (2 cores) for practical runtime. After completion, Spark scores are validated against the custom implementation (max absolute difference reported), top-5 rankings from both implementations are compared side by side, and convergence curves for both are plotted on the same axes.

7. Results and Discussion

7.1 Network Structure

The co-authorship graph constructed from 50,000 arXiv papers is a sparse, scale-free network. The heavy-tailed degree distribution on the log-log plot is a hallmark of real-world collaboration networks and is consistent with preferential-attachment growth [1]. A small number of highly connected authors act as hubs linking different sub-communities, while the majority have only a handful of collaborators.

A large majority of nodes belong to a single giant connected component, consistent with the small-world property. Isolated nodes (single-author papers) form the remaining components and receive only the teleportation contribution to their PageRank scores — a direct consequence of the dead-end handling described in Section 4.

7.2 Top-Ranked Authors

H. Vincent Poor emerges as the highest-ranked author by weighted PageRank in the 50,000-paper subset, followed by Tshilidzi Marwala. These rankings reflect not only the number of collaborators (degree) but also the importance of those collaborators — the recursive nature of PageRank propagates influence through second- and higher-order connections.

The ego-network visualisation of H. Vincent Poor confirms the structural basis for his score: his direct neighbourhood is densely interconnected, meaning that his collaborators themselves have high PageRank scores, which amplifies his own score through the propagation mechanism.

7.3 Degree vs. PageRank Divergence

The log-log scatter plot of degree vs. PageRank score shows a positive but imperfect correlation. A non-trivial fraction of authors exhibit substantial rank differences between the two metrics:

- **High degree, lower-than-expected PageRank:** these authors collaborate widely but mostly with peripheral researchers. Their high edge count does not translate to influence because their neighbours are not themselves influential.
- **Moderate degree, higher-than-expected PageRank:** these authors have fewer collaborators but are connected to other high-PageRank nodes. Their influence propagates recursively through the graph.

This divergence is precisely what motivates link-analysis algorithms over degree-based ranking. A pure degree ranking would miss the second category entirely.

7.4 Convergence Behaviour

The power-iteration algorithm converges monotonically from iteration 1. The L_1 error decreased consistently from 3.8×10^{-1} at iteration 1 to below 10^{-6} by iteration 60. This behaviour is guaranteed by the Perron–Frobenius theorem: because the teleportation term makes the transition matrix stochastic, aperiodic, and irreducible, convergence to the unique stationary distribution is guaranteed from any initialisation (RU 5.1.2).

7.5 Spark Validation

The Spark MapReduce implementation produces scores that differ from the custom single-machine implementation by at most 10^{-6} (mean difference on the order of 10^{-8}), and the top-5 rankings are identical. This confirms that the distributed formulation correctly implements the same mathematical update as the single-machine version.

On the Colab VM (2 cores, local Spark mode) the Spark implementation is slower than the single-machine version due to DataFrame serialisation overhead. This is expected: Spark’s advantage manifests at cluster scale, where data parallelism across many nodes more than compensates for the coordination cost. The implementation is demonstrated on the 50,000-paper subset for reproducibility within the Colab environment; on the full 2-million-paper arXiv snapshot, where the graph exceeds single-machine memory, the Spark version would continue to operate while the single-machine version would fail with an out-of-memory error.

7.6 Limitations and Future Directions

The main limitations of the current work are:

- **Author name disambiguation:** merging name variants (“J. Smith” vs. “John Smith”) would produce a more accurate graph with fewer spurious isolated components.
- **Undirected graph:** co-authorship is symmetric by construction. A directed citation graph would expose dead-end and spider-trap pathologies more naturally and provide an asymmetric influence measure.
- **Subset only:** the 50,000-paper subset may not fully represent the rankings that would result from the full ~ 2 -million-paper dataset.
- **No temporal analysis:** author influence evolves over time; a year-by-year analysis

would reveal how rankings change across decades of scientific publication.

Future work could apply PageRank to the citation graph (directed), integrate author disambiguation using external identifiers such as ORCID, compare PageRank with betweenness and eigenvector centrality, and run the Spark pipeline on the full arXiv snapshot using Google Dataproc.

References

- [1] Barabási, A.-L., & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286(5439), 509–512.
- [2] Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1–7), 107–117.
- [3] Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of Massive Datasets* (3rd ed.). Cambridge University Press. Available at <http://www.mmids.org>.
- [4] Newman, M. E. J. (2001). The structure of scientific collaboration networks. *Proceedings of the National Academy of Sciences*, 98(2), 404–409.